Musanze Harvest Intelligence Service

SWE 3513 Artificial Intelligence - Individual Assignment 2 - Technical report

Full name	TUYISINGIZE Devotha
Registration number	25/27747
Institutional username	ug2527747@ines.ac.rw
Verification code	**AI-A2-7747**
Personal seed (LLLL)	7747
Final code commit	18d2a89a49647a83f546bbfd649600ff660454df
Dataset SHA-256	82120e732b1ac8c8b2acd4d7e70b332ebf2a0e18fe402bcef29a751a94768205

1. Identity and decision problem

Stakeholder

The **field officer of a potato cooperative in Musanze**. Farm records arrive in inconsistent spreadsheets, internet access is unreliable, and the officer has limited time to visit every farm. The officer needs a local tool that turns a farm's growing conditions into evidence for prioritising visits and advice. A small API allows later integration with a mobile app.

Prediction questions

Task	Question for the field officer	Output
Regression	How many tonnes of potatoes is this farm likely to harvest?	yield_tons (t)
Classification	Is this farm at high production risk, i.e. will its yield per hectare be below the cooperative median?	high_risk 0/1 and probability
Clustering	Which group of farms with similar growing conditions does this farm belong to?	cluster ID and plain-language profile

Ethical boundary

This is a **classroom decision-support prototype trained on synthetic data**. It supports, and never replaces, the judgement of the field officer or an agronomist. It must **not** be used to decide access to credit, insurance, subsidies, inputs or any public service, or to rank or penalise farmers. Every prediction must be reviewed by a person who knows the farm before any action is taken. The API and CLI print this advisory with every prediction.

Headline results (held-out test set)

Task	Selected model	Result	Baseline
Regression	linear_regression	MAE 0.726 t, RMSE 0.938 t, R^2 0.865	mean: RMSE 2.594 t
Classification	logistic_regression	F1 0.926, ROC-AUC 0.995	majority: F1 0.000
Clustering	KMeans, K=6	silhouette 0.186	K=2..6 compared

2. Data and reproducibility

All data is **synthetic**, produced by `data/generate_data.py` with `numpy.random.default_rng(7747)`. No CSV is edited by hand.

Data dictionary

Column	Generation rule	Type	Use
rainfall_mm	Uniform 600 to 1400	float	feature
avg_temp_c	Uniform 12 to 26	float	feature
soil_moisture_pct	Uniform 20 to 80; 14 values set missing	float	feature
fertilizer_kg	Uniform 0 to 250; 14 values set missing	float	feature
field_size_ha	Uniform 0.2 to 4.0	float	feature
season	Random A or B	category	feature (one-hot)
cooperative_support	Random 0 or 1	binary	feature
yield_tons	0.004 x rain + 0.06 x soil + 0.02 x fert + 1.30 x size - 0.25 x |temp - 18| + 0.70 x support + season (A +0.50, B -0.20) + N(0, 0.80); clipped at 0.50	float	regression target
high_risk	1 if yield_tons / field_size_ha is below its median, else 0	binary	classification target

Personal seed rule and row count

Rule	Computation	Value
Seed	LLLL = last four digits of 25/27747	7747
Base rows	240 + (7747 mod 41) = 240 + 39	279
Missing per column	m = round(0.05 x 279)	14
Rows written to CSV	279 base rows + 3 exact duplicates	282
Rows after cleaning	duplicates removed, no range violations	279
Train / test	stratified 80/20 split, random_state=7747	223 / 56

Missing-value and duplicate evidence

The validation step (`src/validation.py`) loaded 282 rows and found **14** missing soil-moisture values, **14** missing fertilizer values and no missing values elsewhere. It found **3** exact duplicate rows at CSV positions [279, 280, 281], copies of rows [264, 164, 43]; they were removed, keeping the first copy. All values were inside their valid ranges, every required column was present, and recomputing the high-risk rule from the CSV agreed with the stored label for 100% of farms. Classes are balanced (140 not high risk, 139 high risk). Missing values are kept as NaN and imputed later inside the pipeline, from training rows only.

Fingerprint and reproducibility

The CSV is written with `float_format="%.4f"` and Unix line endings so the bytes, and therefore the hash, are the same on Windows, macOS and Linux. SHA-256 of `data/AI_A2_2527747.csv`:

```
82120e732b1ac8c8b2acd4d7e70b332ebf2a0e18fe402bcef29a751a94768205
```

Running `run_all.py` twice produced byte-identical CSV, JSON metric files and test predictions. The same fingerprint and metrics were also obtained with older library versions (numpy 2.1, pandas 2.2, scikit-learn 1.5). A seed other than 7747, or another student's ID, is rejected.

3. Pipeline design

Flow of `run_all.py`

```
check student ID and seed -> generate CSV + SHA-256 -> validate schema, ranges, missing values,
duplicates and leakage -> remove duplicates -> stratified train/test split (random_state=7747) ->
fit pipelines on train only -> 5-fold cross-validation on train to select models -> score the
held-out test set once -> save models, JSON, CSV and plots.
```

Train/test separation

Duplicates are removed *before* the split, so the same farm can never appear in both sets. One fixed split (223 train, 56 test, stratified on high_risk) is shared by all three tasks. Model selection uses cross-validation inside the training set; the test rows are used only for the final report.

Preprocessing (one ColumnTransformer inside every model Pipeline)

Columns	Steps (fitted on training data only)
5 numeric features	median imputation -> add temp_stress_c = |avg_temp_c - 18| -> standard scaling (scaling skipped for random forests, which do not need it)
season	most-frequent imputation -> one-hot encoding (handle_unknown='ignore')
cooperative_support	most-frequent imputation, kept as 0/1
clustering features	the 5 numeric growing conditions: median imputation -> standard scaling -> KMeans

Because preprocessing and model are saved together as one Pipeline (joblib), `predict.py` and `app.py` apply exactly the same transformations as training. The engineered temperature-stress feature uses an input only; it reflects the agronomic fact that potatoes grow best near 18 °C.

Leakage controls

Risk	Control
Target used as input	Features are fixed to the 7 inputs. yield_tons, high_risk and yield per hectare are on a forbidden list checked by validation.py; a test confirms the saved models see only the 7 inputs.
Statistics from test rows	Imputers, scalers and encoders are fitted inside Pipelines, so in each CV fold and in the final fit they only see training rows.
Hidden proxy of the target	Feature-target correlations are checked; the largest is 0.78, far below 0.98.
Target sent to the API	The request schema forbids extra fields, so yield_tons in a request returns 422.

Model candidates and selection rule

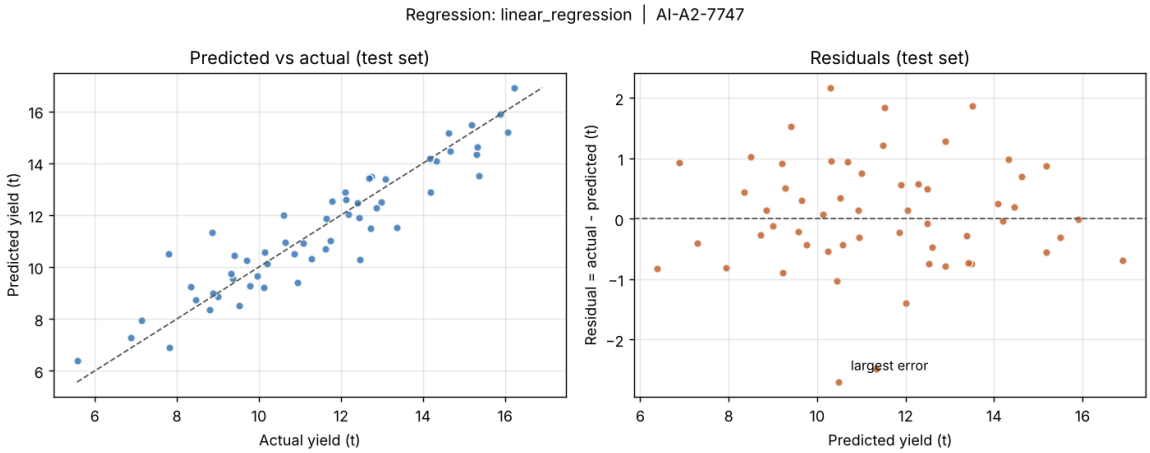
Task	Baseline	Candidates	Selection evidence
Regression	DummyRegressor (mean)	LinearRegression; RandomForestRegressor (300 trees, min_samples_leaf=2)	lowest 5-fold CV RMSE on train
Classification	DummyClassifier (most frequent)	LogisticRegression; RandomForestClassifier (300 trees, min_samples_leaf=2)	highest 5-fold stratified CV F1
Clustering	-	KMeans for K = 2, 3, 4, 5, 6 (n_init=20)	highest silhouette on train

Every random_state is 7747; random forests use n_jobs=1 so results are deterministic.

4. Regression and classification

Regression: yield_tons (56 test farms)

Model	CV RMSE (train)	Test MAE	Test RMSE	Test R^2
mean baseline	2.9480	2.1422	2.5937	-0.0341
linear_regression **(selected)**	0.9594	0.7259	0.9382	0.8647
random_forest	1.3750	0.9853	1.2207	0.7710



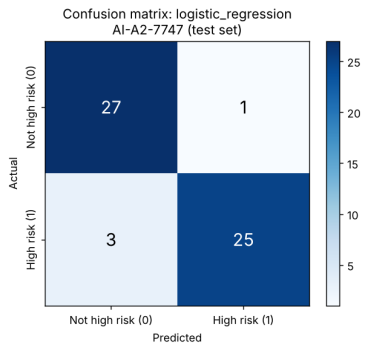
Interpretation. Linear regression was selected because it has the lowest cross-validated RMSE (0.959 t vs 1.375 t). The true relationship is additive, and with 223 training farms the forest cannot learn it as precisely. Residuals are centred on zero (mean +0.066 t) with no visible trend, and 80% of test farms are within 1 t. The test RMSE (0.938 t) is close to the 0.80 t standard deviation of the random noise in the data, so most of the remaining error cannot be predicted from the inputs.

Important regression error (farm 138). Actual 7.79 t, predicted 10.49 t (error -2.70 t). Inputs: rainfall 1375 mm, temp 25.3 C, soil moisture 50.4 %, fertilizer 25 kg, field 1.91 ha, season A, cooperative support 0. The farm has very high rainfall, which pushes the estimate up, but it is hot (about 7 °C above the optimum) and used little fertilizer. The model accounts for these, so the remaining shortfall is mainly the random noise term, about 3.4 noise standard deviations below expectation. A field officer should treat such a gap as a sign of a problem the inputs do not record, such as disease or poor seed.

Classification: high_risk (56 test farms, 28 per class)

Model	CV F1 (train)	Precision	Recall	F1	ROC-AUC
majority baseline	0.0000	0.0000	0.0000	0.0000	not valid*
logistic_regression **(selected)**	0.9814	0.9615	0.8929	0.9259	0.9949
random_forest	0.9188	0.9630	0.9286	0.9455	0.9898

* The majority baseline gives the same probability to every farm, so ROC-AUC is not meaningful (equal to 0.5).



Interpretation. Logistic regression was selected because it has the highest cross-validated F1 (0.981 vs 0.919). On the test set the random forest scored slightly higher (F1 0.945), but the choice was fixed before the test set was used, which keeps the reported metrics honest. The selected model made 1 false alarm and 3 missed high-risk farms out of 56. The strongest input is field size: large fields tend to have lower yield per hectare, so they are flagged more often.

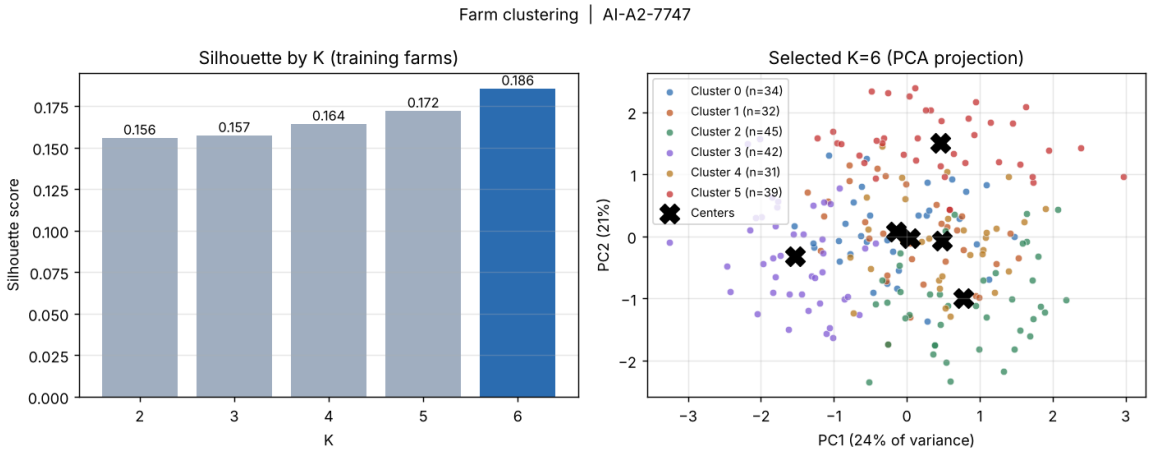
Important classification error (farm 11). False negative (missed high-risk farm): probability 0.36, actual yield 5.89 t/ha against a median of about 6.19 t/ha. Inputs: rainfall 1191 mm, temp 20.2 C, soil moisture 72.7 %, fertilizer 80 kg, field 2.16 ha, season A, cooperative support 0. Rainfall and soil moisture are high, so the model expected a good yield per hectare, but the farm fell just below the median. Misses like this sit near the boundary; a lower threshold would catch them at the cost of more false alarms.

5. Clustering

KMeans was fitted on the five standardised growing-condition features of the 223 training farms. **No target was used to form the clusters**; yield and risk were attached afterwards only to describe them.

K comparison

K	2	3	4	5	6
Silhouette	0.1560	0.1575	0.1644	0.1723	0.1857
Inertia	929.9	804.6	714.2	644.2	576.4



Choice of K. K=6 has the highest silhouette score among K=2..6 (0.1857). The best silhouette (0.186) is modest, which is expected because the synthetic inputs are drawn independently from uniform distributions: farms form a continuum rather than sharply separated groups. The score keeps rising up to the largest K tested, so K is chosen at the edge of the allowed range; the gain per extra cluster is small. The clusters are therefore useful as practical segments for planning field visits, not as natural farm types.

Cluster profiles

Cluster	Farms	Centre (rain mm / temp °C / soil % / fert kg / size ha)	Mean yield	High-risk rate
0	34	942 / 16.2 / 43.6 / 68 / 2.79	11.26 t	91%
1	32	1215 / 16.7 / 37.6 / 174 / 1.32	12.46 t	9%
2	45	809 / 16.1 / 62.9 / 181 / 1.84	12.74 t	38%
3	42	814 / 21.5 / 37.3 / 87 / 1.25	8.18 t	40%
4	31	988 / 22.7 / 52.4 / 188 / 3.04	14.31 t	84%
5	39	1259 / 20.3 / 67.3 / 95 / 2.08	13.40 t	44%

Cluster 0 (34 training farms): low temperature, low fertilizer use, high field size. Average harvest 11.26 t; these farms are high-risk more often than the cooperative average (91% vs 50%).

Cluster 1 (32 training farms): high rainfall, low temperature, low soil moisture, high fertilizer use, low field size. Average harvest 12.45 t; these farms are high-risk less often than the cooperative average (9% vs 50%).

Cluster 2 (45 training farms): low rainfall, low temperature, high soil moisture, high fertilizer use. Average harvest 12.74 t; these farms are high-risk less often than the cooperative average (38% vs 50%).

Cluster 3 (42 training farms): low rainfall, high temperature, low soil moisture, low fertilizer use, low field size. Average harvest 8.18 t; these farms are high-risk less often than the cooperative average (40% vs 50%).

Cluster 4 (31 training farms): high temperature, high fertilizer use, high field size. Average harvest 14.31 t; these farms are high-risk more often than the cooperative average (84% vs 50%).

Cluster 5 (39 training farms): high rainfall, high soil moisture. Average harvest 13.40 t; these farms are high-risk less often than the cooperative average (44% vs 50%).

Operational meaning

Cluster 0 is the clearest example: large fields (about 2.8 ha) with little fertilizer (about 68 kg) on cooler sites. Their total harvest is average (11.3 t) because the fields are big, but 91% of them are high risk because the fertilizer is spread over many hectares. For the field officer this group is a priority for advice on fertilizer rate per hectare. Cluster 3 (hot, dry, small farms with low inputs) has the lowest total harvest and fits water-saving and heat advice. The clusters plan visits; they do not judge individual farmers.

6. API and testing

Endpoint contract (FastAPI, `uvicorn app:app --host 127.0.0.1 --port 8000`)

Endpoint	Behaviour
GET /health	200 {status: ok, models_loaded: true, verification_code: AI-A2-7747}
GET /model-info	200 identity, dataset SHA-256, selected models, test metrics, features, valid ranges, cluster profiles
POST /predict	Body: one FarmRecord (7 typed fields). 200 returns yield_estimate_tons, high_risk_prediction, high_risk_label, high_risk_probability, cluster, cluster_profile, verification_code, advisory

Validation rules (src/schema.py, ranges from src/config.py): rainfall 600-1400, temperature 12-26, soil moisture 20-80, fertilizer 0-250, field size 0.2-4.0, season A/B, cooperative_support 0/1. Types are strict: a string is not converted to a number and true/false is not accepted as 1/0. Unknown fields are refused.

Valid example

```
POST /predict {"rainfall_mm": 1050.0, "avg_temp_c": 18.5, "soil_moisture_pct": 55.0,
"fertilizer_kg": 120.0, "field_size_ha": 1.5, "season": "A", "cooperative_support": 1}
-> 200 {"yield_estimate_tons": 12.9108, "high_risk_prediction": 0, "high_risk_probability": 0.0282,
"cluster": 5, "verification_code": "AI-A2-7747", ...}
```

Invalid example (samples/invalid.json)

```
{"rainfall_mm": 2500.0, "avg_temp_c": "warm", "fertilizer_kg": 120.0, "field_size_ha": 1.5,
"season": "C", "cooperative_support": 1}
-> 422 {"status": "invalid_input", "errors": [{"field": "rainfall_mm", "problem": "Input should be
less than or equal to 1400"}, {"field": "avg_temp_c", "problem": "Input should be a valid number"},
{"field": "soil_moisture_pct", "problem": "Field required"}, {"field": "season", "problem": "Input
should be 'A' or 'B'"}]}
```

Failure behaviour

Situation	CLI (predict.py)	API
Missing field, wrong type, out of range, unknown field	JSON error list on stderr, exit code 2	422 with one entry per bad field
Malformed JSON	exit code 2, 'Malformed JSON'	422
Models not trained yet	exit code 3 with the run_all.py command	503 with the run_all.py command; /health reports degraded
Wrong seed or student ID in run_all.py	exit code 2 with explanation	-

Automated tests (pytest)

Result: **32 passed in 3.62s**. Full output is in evidence/AI_A2_2527747_TEST_LOG.pdf.

File	What is tested
tests/test_inference.py	valid inference returns every field; deterministic prediction; missing field; 4 wrong types; 8 range/category violations; target column in input refused; CLI exit codes for valid and invalid samples
tests/test_data_and_pipeline.py	dataset bytes reproducible and equal to the saved SHA-256; 279/14/3 data signature; duplicate removal and range checks; wrong seed and wrong ID rejected; missing column rejected; saved models use only the 7 inputs; metric files have the required content
tests/test_api.py	/health, /model-info, valid /predict; 422 for missing field, wrong type plus range, and malformed JSON

Most tests exercise failure paths, as the assessor's checklist requires.

7. Model card and reflection

Item	Description
Intended use	Help a cooperative field officer in Musanze estimate harvests, spot farms that may need support and group farms for visit planning. Classroom prototype only.
Users	Field officers and agronomists who can check the output against what they see on the farm.
Data	Synthetic: 279 farms generated from a known formula with seed 7747. No real farmer is described.
Models	Regression: linear_regression; classification: logistic_regression (threshold 0.5); clustering: KMeans K=6 on growing conditions.
Performance	Test RMSE 0.938 t, R^2 0.865; F1 0.926, ROC-AUC 0.995; silhouette 0.186. These numbers describe the synthetic data and would be lower on real farms.
Limitations	Inputs are independent and uniform, the yield formula is linear, and the noise is normal, which makes the problem easier than reality. 279 farms is a small sample. high_risk is relative (half of the farms are always high risk) and depends heavily on field size. Missing data is only random. Inputs outside the training ranges are rejected, not extrapolated.
Risks	Over-trusting a precise-looking number; labelling large farms as 'risky' without context; unfair treatment if outputs were linked to money or services; outdated models if climate or practices change.
Prohibited use	Deciding credit, insurance, subsidies, input allocation or access to any public service; ranking or penalising farmers; any use as the only basis for a decision.
Human oversight	Every output carries an advisory. A field officer must review it with the farmer before acting, and can override it. Disagreements should be recorded to improve the model.

What I would improve with real data

1. Collect real cooperative records over several seasons, with the farmer's consent, and check data quality (units, outliers, missing values that are not random).

2. Add variables that matter in Musanze: potato variety, seed quality, altitude, slope, planting date, late blight pressure and soil tests.

3. Validate by season and by sector (train on past seasons, test on the next) rather than a random split, and report uncertainty, for example prediction intervals for yield.

4. Define high risk with the cooperative using an absolute yield per hectare that matters to farmers, and choose the threshold by the cost of missing a farm versus a false alarm.

5. Check fairness across farm sizes, sectors and support status, and monitor the model after deployment.

Reflection

The project showed that a simple, well-validated model can beat a more complex one when the data is small and the relationship is close to linear, and that the selection rule must be fixed before looking at the test set. The most useful lessons were about engineering: one command that rebuilds everything, the same pipeline for training and inference, and clear errors when input is wrong.